2.2 게이트 타입 분석

개요

PLONK 코드베이스는 다양한 타입의 게이트를 지원하여 효율적인 회로 구성을 가능하게 합니다. 각 게이트는 특정한 제약 조건을 표현하며, `GateConstraint` 트레이트를 구현하여 일관된 인터페이스를 제공합니다.

1. 산술 게이트 (Arithmetic Gate)

구현 위치

- `plonk-core/src/proof_system/widget/arithmetic.rs`

구조 및 역할

산술 게이트는 PLONK의 가장 기본적인 게이트로, 다음과 같은 일반적인 산술 제약 조건을 표현합니다:

```
q_m * a * b + q_l * a + q_r * b + q_o * c + q_4 * d + q_hl * a^α + q_hr * b^α +
```

여기서:

- `a`, `b`, `c`, `d`: 와이어 값들
- `q_m`, `q_l`, `q_r`, `q_o`, `q_4`, `q_c`: 선택자 다항식들
- `q_hl`, `q_hr`, `q_h4`: 고차 선택자 다항식들 (S-box 연산용)
- `α`: S-box 지수 (SBOX_ALPHA = 5)

주요 구성 요소

ProverKey 구조체

```
pub struct ProverKey<F> {
pub q_m: (DensePolynomial<F>, Evaluations<F>), // 곱셈 선택자
pub q_l: (DensePolynomial<F>, Evaluations<F>), // 왼쪽 와이어 선택자
pub q_r: (DensePolynomial<F>, Evaluations<F>), // 오른쪽 와이어 선택자
pub q_o: (DensePolynomial<F>, Evaluations<F>), // 출력 와이어 선택자
pub q_4: (DensePolynomial<F>, Evaluations<F>), // 네 번째 와이어 선택자
pub q_c: (DensePolynomial<F>, Evaluations<F>), // 상수 선택자
pub q_hl: (DensePolynomial<F>, Evaluations<F>), // 고차 왼쪽 선택자
pub q_hr: (DensePolynomial<F>, Evaluations<F>), // 고차 오른쪽 선택자
pub q_h4: (DensePolynomial<F>, Evaluations<F>), // 고차 네 번째 선택자
pub q_arith: (DensePolynomial<F>, Evaluations<F>), // 산술 게이트 활성화 선택자
}
```

핵심 메서드

- `compute_quotient_i()` : 특정 인덱스에서 몫 다항식에 대한 기여도 계산
- `compute_linearisation()` : 선형화 다항식에 대한 기여도 계산

사용 예시

```
composer.arithmetic_gate(|gate| {
gate.witness(a, b, Some(c))
```

```
.add(F::one(), F::one()) // a + b = c

.pi(public_input)

});
```

2. 범위 게이트 (Range Gate)

구현 위치

- `plonk-core/src/proof_system/widget/range.rs`
- `plonk-core/src/constraint_system/range.rs`

구조 및 역할

범위 게이트는 변수가 특정 비트 범위 내에 있음을 증명하는 데 사용됩니다. 4진법 분해를 통해 효율적으로 구현됩니다.

제약 조건

범위 게이트는 다음과 같은 델타 함수를 사용합니다:

```
δ(x) = x(x-1)(x-2)(x-3)
```

이 함수는 x가 {0, 1, 2, 3} 중 하나일 때만 0이 됩니다.

Range 구조체

```
pub struct Range<F>(PhantomData<F>)

where
```

```
F: PrimeField;


impl<F> GateConstraint<F> for Range<F> {
type CustomVals = RangeVals<F>;
fn  constraints(
separation_challenge: F,
wit_vals: WitnessValues<F>,
custom_vals: Self::CustomVals,
) -> F {
let  four = F::from(4u64);
let  kappa = separation_challenge.square();
let  kappa_sq = kappa.square();
let  kappa_cu = kappa_sq * kappa;
let  b_1 = delta(wit_vals.c_val - four * wit_vals.d_val);
let  b_2 = delta(wit_vals.b_val - four * wit_vals.c_val) * kappa;
let  b_3 = delta(wit_vals.a_val - four * wit_vals.b_val) * kappa_sq;
let  b_4 = delta(custom_vals.d_next_val - four * wit_vals.a_val) * kappa_cu;
(b_1 + b_2 + b_3 + b_4) * separation_challenge
}
}
```

사용 예시

```
composer.range_gate(witness, 32); // witness가 32비트 범위 내에 있음을 증명
```

3. 논리 게이트 (Logic Gate)

구현 위치

- `plonk-core/src/proof_system/widget/logic.rs`
- `plonk-core/src/constraint_system/logic.rs`

구조 및 역할

논리 게이트는 XOR 및 AND 연산을 효율적으로 구현합니다. 4진법 분해를 사용하여 비트 연산을 수행합니다.

Logic 구조체

```
pub struct Logic<F>(PhantomData<F>)
where
F: PrimeField;

impl<F> GateConstraint<F> for Logic<F> {
type CustomVals = LogicVals<F>;
fn constraints(
separation_challenge: F,
wit_vals: WitnessValues<F>,
custom_vals: Self::CustomVals,
```

```
) -> F {
let kappa = separation_challenge.square();
let kappa_sq = kappa.square();
let kappa_cu = kappa_sq * kappa;
let kappa_qu = kappa_cu * kappa;

let a = custom_vals.a_next_val - four * wit_vals.a_val;
let c_0 = delta(a);

let b = custom_vals.b_next_val - four * wit_vals.b_val;
let c_1 = delta(b) * kappa;

let d = custom_vals.d_next_val - four * wit_vals.d_val;
let c_2 = delta(d) * kappa_sq;

let w = wit_vals.c_val;
let c_3 = (w - a * b) * kappa_cu;

let c_4 = delta_xor_and(a, b, w, d, custom_vals.q_c_val) * kappa_qu;

(c_0 + c_1 + c_2 + c_3 + c_4) * separation_challenge
}
```

```
}
```

핵심 함수

- `delta()` : 4진법 분해 검증 함수
- `delta_xor_and()` : XOR/AND 연산 검증 함수

사용 예시

```
composer.xor_gate(a, b, num_bits); // XOR 연산

composer.and_gate(a, b, num_bits); // AND 연산
```

4. 기타 게이트 타입

4.1 불린 게이트 (Boolean Gate)

구현 위치

- `plonk-core/src/constraint_system/boolean.rs`

역할

변수가 0 또는 1의 값만 가질 수 있도록 제약합니다.

제약 조건

```
a * (a - 1) = 0
```

사용 예시

```
composer.boolean_gate(variable); // variable이 0 또는 1임을 보장
```

4.2 타원 곡선 게이트 (ECC Gates)

4.2.1 곡선 덧셈 게이트 (Curve Addition Gate)

구현 위치: `plonk-core/src/proof_system/widget/ecc/curve_addition.rs`

역할: 타원 곡선 상의 두 점의 덧셈을 검증합니다.

제약 조건: Twisted Edwards 곡선의 덧셈 공식을 구현

$$x_3 = (x_1 y_2 + y_1 x_2) / (1 + dx_1 x_2 y_1 y_2)$$

$$y_3 = (y_1 y_2 - ax_1 x_2) / (1 - dx_1 x_2 y_1 y_2)$$

4.2.2 고정 기저 스칼라 곱셈 게이트 (Fixed Base Scalar Multiplication Gate)

구현 위치: `plonk-core/src/proof_system/widget/ecc/fixed_base_scalar_mul.rs`

역할: 고정된 기저점에 대한 스칼라 곱셈을 효율적으로 구현합니다.

특징:

- 비트 분해를 통한 스칼라 곱셈
- 누적기(accumulator) 기반 구현
- 비트 일관성 검사 포함

4.3 룩업 게이트 (Lookup Gate)

구현 위치

- `plonk-core/src/proof_system/widget/lookup.rs`
- `plonk-core/src/constraint_system/lookup.rs`

역할

미리 정의된 테이블에서 값을 조회하는 연산을 구현합니다. 복잡한 함수를 효율적으로 표현할 수 있습니다.

구조

```
pub struct ProverKey<F> {
pub q_lookup: (DensePolynomial<F>, Evaluations<F>),
pub table_1: MultiSet<F>,
pub table_2: MultiSet<F>,
pub table_3: MultiSet<F>,
pub table_4: MultiSet<F>,
}
```

특징

- 다중 집합(MultiSet) 기반 구현
- Plookup 프로토콜 사용
- 4개 컬럼까지 지원

게이트 통합 및 사용

GateConstraint 트레이트

모든 게이트는 공통 인터페이스인 `GateConstraint` 트레이트를 구현합니다:

```
pub trait GateConstraint<F>
```

```
where
F: PrimeField,
{
type CustomVals: CustomValues<F>;
fn constraints(
separation_challenge: F,
wit_vals: WitnessValues<F>,
custom_vals: Self::CustomVals,
) -> F;
fn quotient_term(/* ... */) -> F { /* ... */ }
fn linearisation_term(/* ... */) -> DensePolynomial<F> { /* ... */ }
}
```

StandardComposer에서의 게이트 사용

`StandardComposer` 는 다양한 게이트를 통합하여 회로를 구성합니다:

```
impl<F, P> StandardComposer<F, P> {
pub fn arithmetic_gate<Closure>(&mut self, closure: Closure) -> Variable;
pub fn range_gate(&mut self, witness: Variable, num_bits: usize);
pub fn boolean_gate(&mut self, a: Variable) -> Variable;
pub fn xor_gate(&mut self, a: Variable, b: Variable, num_bits: usize) -> Vari
pub fn and_gate(&mut self, a: Variable, b: Variable, num_bits: usize) -> Vari
// ... 기타 게이트 메서드들
```

```
}
```

성능 최적화

1. 선택자 다항식 최적화

각 게이트 타입은 자체 선택자 다항식을 가지며, 이를 통해 특정 위치에서만 활성화됩니다.

2. 배치 처리

여러 게이트의 제약 조건을 배치로 처리하여 성능을 향상시킵니다.

3. 커스텀 게이트

복잡한 연산을 위해 사용자 정의 게이트를 구현할 수 있습니다.

결론

PLONK의 게이트 시스템은 다양한 연산을 효율적으로 표현할 수 있는 유연한 구조를 제공합니다. 각 게이트는 특정한 목적에 최적화되어 있으며, 공통 인터페이스를 통해 일관된 방식으로 통합됩니다. 이러한 설계는 복잡한 회로를 효율적으로 구성하고 증명할 수 있게 해줍니다.